

Google Cloud Analytics Platform

Architecture Design and Bitcoin Cash Pipeline Evidence

Solution to the Astrafy Data Engineer Take-Home Challenge

Daniel Bostan

daniel.bostan@proton.me

github.com/Xocket/astrafy-bch-infra github.com/Xocket/astrafy-bch-dbt

September 2026

Contents

1	AI-assisted development and engineering method	2
1.1	Tools and what each did	2
1.2	What stayed human	2
1.3	Guardrails	2
2	Executive decision	2
3	Part 1: architecture on Google Cloud	3
3.1	Projects, network, and environments	5
3.2	Component choices	5
3.3	Ingestion per source	5
3.4	Data layers and standards	6
4	Delivery: GitOps and data quality	6
5	BI for departments and ML reuse	6
5.1	BI	6
5.2	ML	7
6	Security, privacy, and reliability	7
6.1	Identity and access	7
6.2	Secrets and encryption	7
6.3	Privacy	7
6.4	Reliability and recovery	7
7	Cost and licensing	8
7.1	Cost control	8
7.2	Licensing	8
8	Alternatives, risks, and rollout	9
9	Part 2: the Bitcoin Cash pipeline, as built	10
9.1	Infrastructure (Terraform)	10
9.2	dbt models	10
9.3	Live results	10
9.4	Semantics	11
9.5	Tests	11
9.6	CI and authentication	11
9.7	Honest status	12
	References	13

1 AI-assisted development and engineering method

This project was built as an AI-assisted pair-programming exercise. This section records which assistants did what, and which work stayed human. The rule was the same everywhere: the assistant proposes, the human decides, and a machine gate verifies.

1.1 Tools and what each did

- **Roadmap.** Astra planned the execution: the phase order, the risk list (cloud onboarding, balance semantics, CI authentication), and the time budget.
- **Decisions.** GPT Sol 5.6 analysed each ambiguity in the brief and drafted the decision records in docs/adr/. I selected the interpretation in every case.
- **First code.** GPT Luna 5.6 wrote the first Terraform stack and the first dbt models.
- **Remaining code and tests.** DeepSeek V4.1 Flash wrote the rest of the models, the eight singular tests, and the generic test declarations. I changed assistant mid-project when one subscription ran out, and continued with the same gates.
- **Documentation.** GPT Sol 5.6 drafted the report, the READMEs, and the evidence documents from the repository files and the recorded decisions.
- **Final pass.** GLM 5.3, through the local OpenCode harness, shortened this report, condensed the repository documentation, and checked that every number matches the recorded evidence.

1.2 What stayed human

- **Interpretation.** The brief leaves “current balance”, “Coinbase”, and the three-month window open. I chose each interpretation and recorded it in a decision record.
- **Verification.** I ran the Terraform applies, the dbt build, the tests, and the CI myself. A green result was never assumed.
- **Security.** I reviewed the IAM scopes and the Workload Identity Federation conditions line by line.
- **This text.** I read and approved every part of it.

1.3 Guardrails

AI output was treated as a draft. These gates ran on every change: dbtparse and dbtcompile offline, the 45 dbt tests against BigQuery, terraformfmt and validate with a plan-only CI job, pre-commit with a private-key scan, and a byte cap on every BigQuery job (100 GiB locally, 50 GiB in CI). Every measured number in this report comes from a run log or a repository file, not from an assistant.

2 Executive decision

Recommendation. Build the platform on Google Cloud in separate development and production projects. Capture every source into an immutable raw layer. Transform the data with dbt Core in BigQuery. Publish one governed semantic layer for all dashboards. Reuse the same governed features for model training and model serving. Keep all infrastructure and transformation code in Git: pull requests carry the evidence, humans approve each promotion.

Driver	Architectural answer
Six different sources	One ingestion contract per source: change data capture for databases, events where the source publishes them, checkpointed APIs otherwise.
Several departments	Conformed entities and one semantic layer. Each department gets its own view of the shared metrics.
BI and ML together	Features come from governed intermediate data, never from dashboard tables.
Open source where practical	dbt Core, Debezium, Airflow, Feast, OpenLineage. Managed services stay where they remove real operating work.
GitOps and DataOps	Git is the source of truth. CI produces evidence; protected branches own promotion.
Security and privacy	Separate projects, keyless workload identities, classified data, deletion by design.
Cost	Partition, cluster, dry-run, cap, budget. The free tier is a guardrail, not a promise.

Non-goals.

- Replace operational systems. The design ingests them; it does not migrate them.
- Let BI users query production databases directly.
- Treat budget alerts as a spending cap. Google budgets notify; they do not block.[10]
- Promise exactly-once outcomes. The design assumes at-least-once delivery and makes processing idempotent.[6]

Acceptance. A pilot is accepted when the source-to-raw reconciliation repeats, freshness and completeness targets pass for a representative period, contract tests block bad changes, the Terraform plan shows only reviewed changes, workload identities are keyless and ref-constrained, and cost and scans stay inside approved limits.

3 Part 1: architecture on Google Cloud

The customer data comes from PostgreSQL, MySQL, MongoDB, SAP, Salesforce, and SurveyMonkey. The goal is a platform that serves dashboards for several departments and feeds models for recommendations and predictions. Figure 1 shows the flow: capture, immutable raw storage, warehouse layers, and consumers.

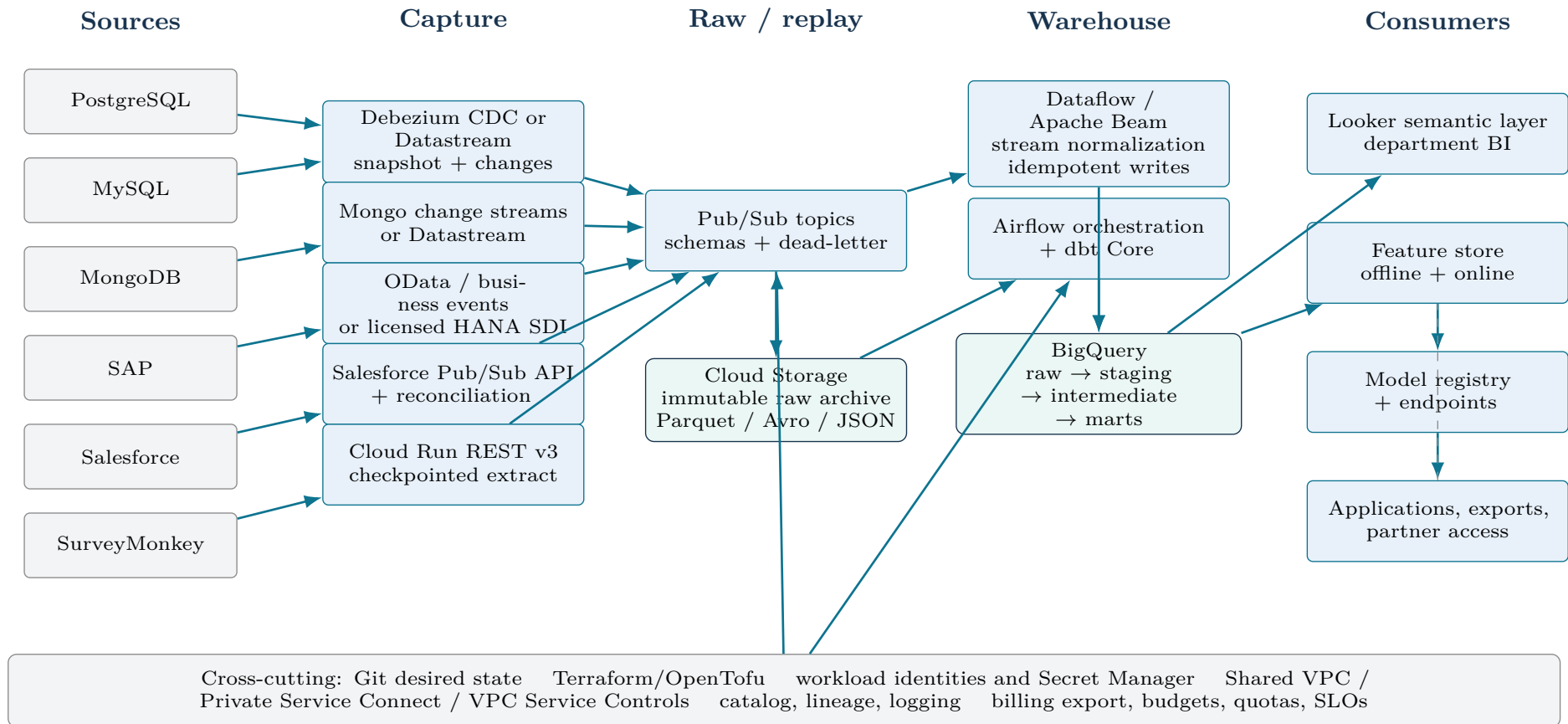


Figure 1: Proposed platform. Everything above the grey bar is data flow; everything in it applies to all stages.

3.1 Projects, network, and environments

- **Folders.** Platform, shared-security, shared-services, data-development, data-production. Folder-level IAM and organization policies apply at each level.
- **Shared VPC.** One host project owns subnets, NAT, DNS, and firewall policy. Data projects attach as service projects.
- **Separate environments.** Development and production are separate projects with separate service accounts, secrets, state prefixes, and workload identity trust. Production data is not readable from development.
- **Private paths.** Private Service Connect and Private Google Access for managed endpoints. Identity-Aware Proxy for administrative access. Controlled egress only.
- **Service perimeter.** VPC Service Controls around BigQuery, storage, and secrets: first in dry-run mode, then enforced. The perimeter and IAM are complementary controls.[8]

3.2 Component choices

Capability	Selected	Reason
Infrastructure as code	Terraform; OpenTofu when policy requires an OSI license	Mature GCP provider. Terraform is BUSL source-available; OpenTofu is MPL-2.0.[26, 27]
Database CDC	Debezium; Datastream where managed is preferred	Debezium is portable open source; Datastream covers PostgreSQL, MySQL, MongoDB, Salesforce.[19, 1]
Events	Pub/Sub; Kafka when replay, ordering, or scale demand it	Pub/Sub needs no cluster. Idempotent processing is mandatory either way.[6]
Raw archive	Cloud Storage immutable objects + BigQuery raw	Portable replay and low-cost retention.
Transformation	dbt Core	Reviewable SQL with tests, contracts, docs, and lineage metadata.[34]
Orchestration	Airflow; Composer when managed operations pay	Open source and DAG-as-code.[33]
Warehouse	BigQuery: raw, staging, intermediate, marts	Partitioned, clustered, governed.
BI	Looker semantic layer	Metrics defined once, governed access.[12, 13]
ML	Feature store, model registry, endpoints (Vertex AI); Feast as the portable option	Reuse of governed features for training and serving.[14, 25]
Lineage and metrics	OpenLineage, Marquez, Cloud Logging/Monitoring	Portable lineage events; managed metrics and alerts.[36]

3.3 Ingestion per source

Every connector states its change signal, its checkpoint, and its delete rule. “Connected” is not enough: each connector has a lag target from source to raw, raw to typed, typed to mart.

Source	Capture	Points that matter
PostgreSQL	Debezium logical decoding (WAL); Datastream alternative	Snapshot plus LSN checkpoint. Least-privilege replication role. Do not infer a delete from a missing row.
MySQL	Debezium row-based binlog; Datastream alternative	GTID or file/position checkpoint. Monitor binlog retention and connector lag.
MongoDB	Change streams from a replica set	Resume-token checkpoint. Keep tombstones and schema versions. Do not flatten nested documents without a rule.
SAP	OData or business events; HANA SDI where licensed	Keep document ID, version, validity times, and cancellation flags. SAP interfaces are proprietary products.[22, 23]
Salesforce	Pub/Sub API change events plus REST reconciliation	Replay-ID checkpoint. Soft deletes and undeletes need the audit extract.[20]
SurveyMonkey	Cloud Run scheduled REST v3 extract	Page cursor per survey. Webhooks wake the job; they are not the payload.[21]

Connector rules.

- Assume at-least-once delivery. Stable keys and deterministic upserts make replay safe.

- Schema changes are explicit: a new revision, compatible by default, quarantined when not.
- Deletes are data: raw keeps the operation log; staging applies the current-state rule.
- Snapshot and stream overlap is safe: deduplicate by source key and version, then reconcile.
- Reconcile counts and checksums on a schedule, not only after failures.

3.4 Data layers and standards

Layer	Contents	Rules
Raw	Exact source payload, operation, offset, ingest time, schema revision	Append-only. Ingestion identities only. No corrections.
Staging	Typed, deduplicated, source-shaped records	Source-level tests. No department logic.
Intermediate	Conformed entities, slowly changing dimensions, feature history	Enterprise identity rules. Owner and contract required.
Marts	Department fact and dimension tables	Versioned, tested, certified, declared as exposures.
Semantic	Metrics, dimensions, joins, access rules	One definition per metric, with owner, grain, and compatibility policy.

Standards.

- Keep the source ID. Add surrogate keys only in the intermediate and mart layers.
- Store UTC timestamps plus the business time zone where the calendar matters.
- Store money as decimals with an explicit currency. Never binary floating point for financial facts.
- Keep event time, commit time, ingest time, and publication time separate.
- Define deletion once for raw, aggregates, features, exports, and backups. A table drop is not a privacy design.

4 Delivery: GitOps and data quality

Git is the desired state for infrastructure, models, semantic definitions, and features. A merge is not a release. Pull requests carry the change and its evidence; protected branches own the promotion.

Pull-request evidence. Format, lint, secret, and policy checks; dbtparse and compile with unit tests; a Terraform plan with a policy and cost diff; a sandbox build with model tests, a data diff, and a scan estimate.

Promotion gates. Infrastructure needs an approved plan and a post-apply smoke test. A connector needs source-owner approval, a network path, and a canary offset. A dbt model needs a full build, a consumer impact check, and a cost check. A semantic change needs owner certification and a dashboard check. A model needs registry approval and a canary endpoint. Rollback is part of every plan: revert the release, pause schedules, replay from the checkpoint, reconcile.

Contracts. A dataset is production-ready when its schema, grain, owner, quality thresholds, freshness target, compatibility policy, and deletion rule are written down. dbt contracts enforce the declared shape before the build; tests check the content after it.[17]

Quality. Tests cover validity, completeness, uniqueness, consistency, freshness, and accuracy. Blocking tests fail the build. Non-blocking tests open an incident with the data product owner.

Lineage. Connector runs emit source, offset range, and schema revision. The dbt manifest maps models to tests, owners, and exposures. OpenLineage events land in Marquez; a managed catalog is optional enrichment.[24, 18]

Incidents. Detect (freshness, reconciliation, contract, or cost alert). Contain (pause the schedule, quarantine bad messages, keep the evidence). Correct (fix the code or contract, replay from the last trusted checkpoint). Reconcile, republish, and add a regression test.

5 BI for departments and ML reuse

5.1 BI

Define each shared metric once in a version-controlled semantic layer. Departments compose their own views from the shared definitions; none of them redefines the metric.[12, 13]

- **Shared foundation:** customer, product, geography, calendar, currency, consent. Centrally owned and certified.
- **Domain semantic:** pipeline, orders, finance, survey response. The domain owner approves grain and definition.
- **Department experience:** dashboards and scheduled delivery under row and field access control. No hidden metric logic.
- **Certified extracts** only for tools that cannot query the semantic layer. Export a stable contract.
- **Query budgets.** Replace repeated expensive logic with governed aggregates.
- **Impact.** Record dashboards as dbt exposures so model changes show their consumers.[18]

5.2 ML

Recommendation and prediction models reuse the warehouse. They do not build private pipelines.

- Features come from governed intermediate data, not from notebooks or dashboard marts.
- A feature definition states entity key, event time, availability time, owner, and version.
- Training sets use point-in-time joins, so a feature is visible only after it existed.[25]
- The same definition and code serve offline training and online serving. Parity and freshness are tested.
- Datasets, code, parameters, artifacts, and approvals are versioned in a model registry.[15]
- Predictions are logged with model version and policy decision, with the minimum personal data.
- Monitoring covers feature drift, prediction drift, latency, segment performance, and feedback quality.[16]
- Feedback enters training only after label maturity and human approval.

6 Security, privacy, and reliability

6.1 Identity and access

- People log in through the corporate identity provider with MFA. Shared accounts are prohibited.
- Workloads use attached service accounts. No service-account keys in Git, CI variables, or images. Workload Identity Federation gives CI short-lived credentials under owner, repository, ref, and actor conditions.[7]
- CI gets only the permission the stage needs. Production roles are separate from development roles.
- Break-glass access is time-bound, approved, logged, and reconciled back to Git.

6.2 Secrets and encryption

External credentials live in Secret Manager with pinned versions and audited access.[9] Cloud KMS customer-managed keys protect high-value datasets and buckets; key administration is separate from data administration. TLS in transit, mTLS where supported. Logs never contain tokens, raw payloads, or personal data.

6.3 Privacy

Classify fields (public, internal, confidential, restricted). Minimize and tokenize identifiers before reuse. Define deletion across raw, aggregates, features, exports, and backups, and test the deletion path before production.

6.4 Reliability and recovery

Tier	RPO	RTO	Recovery method
Tier 0: payments, live recommendations	15 min	4 h	Warm regional capacity; replay from raw.
Tier 1: department dashboards	1 h	8 h	Restore objects, redeploy, rebuild curated layers.
Tier 2: exploration, batch history	24 h	2 days	Rerun from versioned raw.

These targets are proposals; the owners approve them. The design keeps immutable raw data and connector checkpoints, and tests replay. Terraform state lives in a separate versioned backend. IAM, secret references, and policies are recreated from Git after a regional loss. Managed BigQuery disaster recovery is an Enterprise Plus option (about 15-minute replication, five-minute recovery after initiation).[11] Rehearse the critical paths quarterly.

7 Cost and licensing

7.1 Cost control

Google gives each billing account 1 TiB of free on-demand query processing and 10 GiB of free storage per month.[2] This is an allowance for those two services, not a promise about the platform: orchestration, BI, egress, logging, and online ML bill separately.

Control	Implementation
Query design	Partition by date, cluster by common filters, select only the needed columns.[3]
Dry-run gate	Estimate every material build; fail above a set threshold.
Per-job cap	Maximum bytes billed per query: 100 GiB locally and 50 GiB in CI, as used and measured in this project.[4, 5]
Monthly envelope	Alert before 0.8 TiB processed; tune from measured billing data.
Budgets	50/90/100% alerts. Budgets notify; they do not stop spending.[10]
Attribution	Label jobs by project, team, and data product. Export billing data to BigQuery.
Cadence	Weekly top-spend review; monthly unit cost and forecast.

Buy reservations only after four weeks of measured use. Managed disaster recovery, BI, and online feature stores can cost more than the warehouse itself; cost them as first-class services.

7.2 Licensing

Three terms, used consistently. *Open source*: an OSI-approved license. *Source-available*: the source is published, the rights are limited. *Managed*: the vendor runs the service under commercial terms; an open-source SDK does not make the service open.

Component	License	Class and note
PostgreSQL	PostgreSQL License	Open source.[28]
MySQL Community	GPLv2 plus exceptions	Open source; enterprise editions are commercial.[29]
MongoDB Community	SSPL	Source-available, not OSI. Atlas is managed.[30]
Debezium	Apache-2.0	Open source.[31]
Apache Kafka	Apache-2.0	Open source; self-managed clusters are operating work.[32]
Apache Airflow	Apache-2.0	Open source; Composer is managed.[33]
dbt Core	Apache-2.0	Open source; hosted dbt services are managed.[34]
Terraform	BUSL 1.1	Source-available. OpenTofu (MPL-2.0) is the open policy option.[26, 27]
Feast	Apache-2.0	Open source; online stores are operating work.[35]
OpenLineage / Marquez	Apache-2.0	Open source.[36]
Grafana OSS	AGPL-3.0	Open source; Grafana Cloud is managed.[37]
BigQuery, Pub/Sub, Dataflow, Datastream, Looker, Vertex AI	Commercial	Managed services.
SAP, Salesforce, SurveyMonkey interfaces	Commercial	Proprietary APIs and products.

8 Alternatives, risks, and rollout

Area	Selected	Switch when
Warehouse	BigQuery	Portability or concurrency economics dominate.
CDC	Debezium + Datastream option	A source is immutable or low-change: batch is enough.
Events	Pub/Sub	Sustained throughput, long replay, or many consumers justify Kafka.
Streaming	Dataflow / Beam	The team already runs Spark on GKE.
Orchestration	Airflow	Managed operations pay more than flexibility.
BI	Looker	Departments demand Power BI or Tableau; keep the certified contracts either way.
Feature store	Vertex AI	Portability outweighs managed serving (Feast).
IaC	Terraform	Policy requires an OSI license (OpenTofu).

Main risks and responses.

- Source volumes and API rights are unknown until discovery. Measure before committing to Kafka, Dataflow, or reservations.
- SAP and SurveyMonkey depend on licensed interfaces and plan limits. Confirm in the workshop; keep the incremental fallback.
- Metric duplication: without a governed semantic layer, departments silently redefine revenue.
- Feature leakage: without availability timestamps and point-in-time joins, models validate well and fail live.
- The free tier is finite. This project measured about 958 GiB for one full pipeline run (Section 9.3). Caps and budgets control repeats.

Rollout. Discover and contract first: source inventory, owners, classifications, volumes, service targets, and a cost baseline (weeks 0–3). Build the foundation: projects, network, identities, state, and observability (3–7). Pilot one database and one application source end to end (6–11). Expand to the remaining sources and full operations (10–16). Department BI (15–21), then the first bounded ML use case (20–27). Run old and new outputs in parallel for one business cycle. Reconcile totals, grain, and definitions, not just numbers. Migrate one department at a time. Keep rollback until sign-off.

9 Part 2: the Bitcoin Cash pipeline, as built

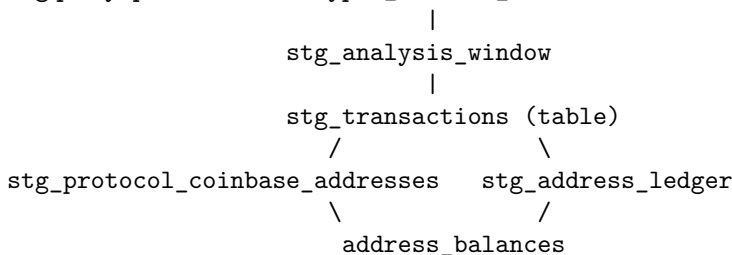
Part 1 is a design. Part 2 is implemented and verified. Everything in this section comes from the two public repositories and from recorded runs.[38, 39] The evidence date is 24 September 2026.

9.1 Infrastructure (Terraform)

- **Bootstrap stack.** A dedicated state project, one private versioned GCS bucket, a read-only plan service account, and an EUR 2 alert budget at 50/90/100%. Bootstrap state is local and gitignored.
- **Primary stack.** Dev project astrafy-thc-006-xocket (number 132073509225), US staging/marts datasets, required APIs, the dbt service account, IAM, and the workload identity pool and provider. State prefix astrafy-bch-analytics/dev in the bootstrap bucket; prod uses a separate prefix and stays un-applied.
- **dbt identity.** Project-level roles/bigquery.jobUser, a custom role with only datasets.create/get, and dataEditor on the two owned datasets. It is not dataEditor on the public dataset.
- **Source view.** The organization cannot change the public dataset IAM, so Terraform creates staging.bch_transactions_source: a view over bigquery-public-data.crypto_bitcoin_cash.transactions that renames the reserved column hash to transaction_hash. The view runs with its creator's access.
- **No static key.** There is no service-account-key resource, no credential variable, and no JSON fallback.
- **CI.** Format and validate run always. An authenticated read-only plan runs on main with workload identity. No workflow can apply.

9.2 dbt models

bch_transactions_source (Terraform view over
bigquery-public-data.crypto_bitcoin_cash.transactions)



- stg_analysis_window captures max(block_timestamp) once and derives the inclusive start at midnight exactly three calendar months earlier. The capture timestamp is audit data; it does not define the window.
- stg_transactions is a table partitioned by month and clustered by hash. Hash deduplication is deterministic. The nested arrays keep only index, addresses, and value.
- stg_protocol_coinbase_addresses collects the non-empty output addresses of transactions with is_coinbase=true in the same window.
- stg_address_ledger turns each address element into its own row: trims whitespace, filters null, empty, and sentinel addresses, keeps the source case, makes inputs negative and outputs positive, and drops the excluded addresses.
- address_balances groups the ledger by address, keeps exact satoshis, and exposes balance_bch=balance_satoshis/100000000.

Every generated table carries workload labels and a seven-day expiration, so abandoned tables cannot bill storage indefinitely.

9.3 Live results

The public source ends on 2024-05-13, so the window is source-relative, not wall-clock. Measured on 24 September 2026:

Measurement	Value
Window	2024-02-13T00:00:00+00:00 to 2024-05-13T23:57:05+00:00, inclusive
Staged transactions	6,495,376
Protocol-coinbase transactions in window	13,230
Excluded protocol-coinbase addresses	4,544
Ledger rows	35,686,444
Mart addresses	5,774,930
Net movement across all mart addresses	6.47 trillion satoshis
Addresses with a negative balance	about 1.06 million (valid; see semantics)
Table storage	12.54 GiB logical; seven-day expiration; labels verified
Query scan, full run	about 958 GiB; stg_transactions alone: 945.6 GiB
Latest-timestamp query estimate	2.91 GiB (3,127,792,896 bytes)
Three-month profile estimate	6.19 GiB (6,646,559,904 bytes)
Tests	45 passed, 0 failed, 0 warned

9.4 Semantics

- **Balance** is the three-month net movement: outputs minus inputs inside the captured window. It is not a lifetime balance. A negative value is valid: the funding can be older than the window.
- **Coinbase** is the protocol coinbase flag, not the exchange. No authoritative exchange-address list exists in the source; the interpretation is deterministic and recorded (ADR 0003).
- **Addresses** are trimmed, not lower-cased. Base58 and Bech32 case is not safely interchangeable.
- **Multisig** outputs credit each listed address. This is attribution, not beneficial ownership.
- **Fees** stay on the transaction and are checked per transaction; they are never assigned to an address.

9.5 Tests

45 tests ran against the live tables: 37 generic (not-null, unique, accepted values, window bounds) and 8 singular assertions:

- `assert_captured_window` — the window is exactly three calendar months, inclusive, and the staged maximum equals the captured maximum.
- `assert_stg_transactions_within_window` — no staged row outside the window or its partition bounds.
- `assert_transaction_accounting` — for complete non-coinbase rows, $\text{input} = \text{output} + \text{fee}$.
- `assert_protocol_coinbase_addresses` — the exclusion set equals the non-empty coinbase output addresses in the window.
- `assert_protocol_coinbase_exclusion` — excluded addresses appear in neither the ledger nor the mart.
- `assert_no_sentinel_addresses` — null, empty, and textual-null sentinels are absent.
- `assert_ledger_entry_uniqueness` — no duplicate ledger rows.
- `assert_balance_equation` — signed balance equals outputs minus inputs, and the BCH conversion is exact.

All 45 passed in the local full run and in CI (pull request and main).

9.6 CI and authentication

- Triggers: pull request opened, synchronized, or reopened, and push to main. Superseded runs cancel.
- A credential-free job runs `dbtparse--no-partial-parse` on every trigger.
- The authenticated job uses workload identity federation to the Terraform-provisioned service account. Both jobs passed in run [36054350409](#).
- Steps: `dbtcompile--no-introspect`, a cost-capped `dbtrun--selectaddress_balances`, `dbttest`, and `dbtdocsgenerate`. Artifacts upload for 14 days.
- The full five-model refresh is manual: the measured 945.6 GiB source scan is close to the 1 TiB monthly allowance.

- The workload identity CEL conditions pin owner, repository, ref, event, and actor. Infrastructure CI is main-only; dbt development trust also accepts owner-initiated pull requests; production accepts dbt main only. Bindings are exact principal sets.
- The infrastructure plan job authenticated with workload identity and found no drift: [run 36054422871](#).

9.7 Honest status

Claim	Evidence	Status
Datasets, service account, WIF exist	Terraform outputs; inspection	Verified
Bootstrap and primary stacks applied	Applied; outputs recorded	Verified
Five dbt models, 45 tests, docs	Local full run; CI run 36054350409	Verified
Credential-free parse	Same CI run 36054350409	Verified
Public repositories	Both GitHub URLs	Verified
Full workload inside the free tier	One full run about 958 GiB; caps and budget in place	Guardrail, not a guarantee
Billing reconciliation	BigQuery billing export not yet reviewed	Open
Human review and submission	Pending recipient name	Open

Two gates remain human: reconcile the BigQuery billing export for the measured run, and review this PDF, attach it, and send the reply email.

References

All web sources were accessed on 24 September 2026. Vendor documentation and prices can change; verify them again at procurement time.

References

- [1] Google Cloud, *Datastream overview*. <https://docs.cloud.google.com/datastream/docs/overview>
- [2] Google Cloud, *BigQuery pricing*. <https://cloud.google.com/bigquery/pricing>
- [3] Google Cloud, *Introduction to partitioned tables*. <https://docs.cloud.google.com/bigquery/docs/partitioned-tables>
- [4] Google Cloud, *Estimate and control costs*. <https://docs.cloud.google.com/bigquery/docs/best-practices-costs>
- [5] Google Cloud, *Create custom query quotas*. <https://docs.cloud.google.com/bigquery/docs/custom-quotas>
- [6] Google Cloud, *Exactly-once delivery*. <https://docs.cloud.google.com/pubsub/docs/exactly-once-delivery>
- [7] Google Cloud, *Workload Identity Federation*. <https://docs.cloud.google.com/iam/docs/workload-identity-federation>
- [8] Google Cloud, *Overview of VPC Service Controls*. <https://docs.cloud.google.com/vpc-service-controls/docs/overview>
- [9] Google Cloud, *Secret Manager best practices*. <https://docs.cloud.google.com/secret-manager/docs/best-practices>
- [10] Google Cloud, *Create, edit, or delete budgets and budget alerts*. <https://docs.cloud.google.com/billing/docs/how-to/budgets>
- [11] Google Cloud, *Managed disaster recovery*. <https://docs.cloud.google.com/bigquery/docs/managed-disaster-recovery>
- [12] Google Cloud, *Introduction to LookML*. <https://docs.cloud.google.com/looker/docs/what-is-lookml>
- [13] Google Cloud, *Looker access control and permission management*. <https://docs.cloud.google.com/looker/docs/access-control-and-permission-management>
- [14] Google Cloud, *Feature Store overview*. <https://docs.cloud.google.com/gemini-enterprise-agent-platform/machine-learning/featurestore/latest/overview>
- [15] Google Cloud, *Introduction to Model Registry*. <https://docs.cloud.google.com/vertex-ai/docs/model-registry/introduction>
- [16] Google Cloud, *Introduction to Model Monitoring*. <https://docs.cloud.google.com/vertex-ai/docs/model-monitoring/overview>
- [17] dbt Labs, *Model contracts*. <https://docs.getdbt.com/docs/mesh/govern/model-contracts>
- [18] dbt Labs, *Add exposures to your DAG*. <https://docs.getdbt.com/docs/build/exposures>
- [19] Debezium, *Source connectors*. <https://debezium.io/documentation/reference/stable/connectors/index.html>
- [20] Salesforce Developers, *Subscribe with Pub/Sub API*. <https://developer.salesforce.com/docs/platform/change-data-capture/guide/cdc-subscribe-pubsub-api.html>
- [21] SurveyMonkey, *SurveyMonkey API v3 documentation*. <https://api.surveymonkey.com/v3/docs>
- [22] SAP Help Portal, *Events on SAP Business Accelerator Hub*. https://help.sap.com/docs/SAP_S4HANA_CLOUD/0f69f8fb28ac4bf48d2b57b9637e81fa/8cbf952e55364254be2da77aa1342aa5.html
- [23] SAP Help Portal, *SAP HANA Smart Data Integration overview*. https://help.sap.com/docs/HANA_SMART_DATA_INTEGRATION/b2f0282053648f8a1ef873e65ded81a/323ff4c3c12040bab8f1222a901dd95d.html
- [24] OpenLineage, *dbt integration*. <https://openlineage.io/docs/integrations/dbt/>
- [25] Feast, *Introduction: open-source offline/online feature store*. <https://docs.feast.dev/>
- [26] HashiCorp, *Terraform LICENSE* (BUSL 1.1 for current releases). <https://raw.githubusercontent.com/hashicorp/terraform/main/LICENSE>
- [27] OpenTofu, *LICENSE* (MPL-2.0). <https://raw.githubusercontent.com/opentofu/opentofu/main/LICENSE>
- [28] PostgreSQL Global Development Group, *PostgreSQL License*. <https://www.postgresql.org/about/licence/>
- [29] Oracle MySQL, *MySQL 8.4 Licensing Information* (Community GPLv2 plus exceptions). <https://raw.githubusercontent.com/mysql/mysql-server/8.4/LICENSE>
- [30] MongoDB, *Server Side Public License*. <https://www.mongodb.com/legal/licensing/server-side-public-license>
- [31] Debezium, *LICENSE* (Apache License 2.0). <https://raw.githubusercontent.com/debezium/debezium/main/LICENSE.txt>
- [32] Apache Kafka, *LICENSE* (Apache License 2.0). <https://raw.githubusercontent.com/apache/kafka/trunk/LICENSE>
- [33] Apache Airflow, *LICENSE* (Apache License 2.0). <https://raw.githubusercontent.com/apache/airflow/main/LICENSE>
- [34] dbt Core, *LICENSE* (Apache License 2.0). <https://raw.githubusercontent.com/dbt-labs/dbt-core/main/LICENSE>
- [35] Feast, *LICENSE* (Apache License 2.0). <https://raw.githubusercontent.com/feast-dev/feast/master/LICENSE>
- [36] Marquez, *LICENSE* (Apache License 2.0). <https://raw.githubusercontent.com/ MarquezProject/marquez/main/LICENSE>
- [37] Grafana, *LICENSE* (AGPL-3.0 for Grafana OSS). <https://raw.githubusercontent.com/grafana/grafana/main/LICENSE>
- [38] Infrastructure repository: <https://github.com/Xocket/astrafy-bch-infra>
- [39] dbt and CI repository: <https://github.com/Xocket/astrafy-bch-dbt>